

Tree-Based Learning Methods: Decision Tree, Bagging, and Random Forest on the Adult Dataset

Melvin Gates Jr.

April 2026

1 Introduction

Tree-based learning methods are widely used in machine learning because they can model nonlinear relationships and interactions among features. In this project, three supervised classification methods were compared: a single Decision Tree, Bagging with decision trees as base learners, and Random Forest.

The purpose of this study was to evaluate how these methods differ in predictive performance, generalization ability, computational cost, robustness, and interpretability on a real-world dataset. Special attention was given to overfitting, variance reduction, and ensemble diversity.

2 Dataset Description

The dataset used in this study is the Adult (Census Income) dataset obtained from the UCI Machine Learning Repository. The dataset contains 48,842 instances and 14 input features, including both categorical and numerical variables.

The prediction task is binary classification: determining whether an individual's annual income exceeds \$50,000 based on census-derived attributes. The dataset includes variables such as age, workclass, education, marital status, occupation, relationship, race, sex, capital gain, capital loss, hours per week, and native country.

Some features contain missing values represented by the symbol "?". These values were handled during preprocessing.

3 Preprocessing Pipeline

The preprocessing pipeline included handling missing values, encoding categorical variables, and preparing the data for model training.

Missing values represented by “?” were replaced with null values. Numerical features were imputed using the median, while categorical features were imputed using the most frequent value.

Categorical variables were transformed using one-hot encoding so that they could be used by the machine learning models.

Feature scaling was not applied because tree-based models are generally insensitive to feature scaling since they rely on threshold-based splitting rather than distance-based calculations.

The dataset was split into training and testing sets using an 80/20 split. Hyperparameter tuning was performed using cross-validation on the training set only in order to maintain a fair evaluation protocol.

4 Methodology

4.1 Decision Tree

A decision tree recursively partitions the feature space by selecting the feature and threshold that minimize impurity. A common impurity measure is the Gini index:

$$G(D) = 1 - \sum_{k=1}^K p_k^2$$

where p_k is the proportion of class k in node D .

If a split divides the data into left and right child nodes, the weighted impurity after the split is:

$$G_{\text{split}} = \frac{|D_{\text{left}}|}{|D|} G(D_{\text{left}}) + \frac{|D_{\text{right}}|}{|D|} G(D_{\text{right}})$$

The best split is the one that minimizes G_{split} . Decision trees are easy to interpret, but they can overfit the training data and have high variance.

4.2 Bagging

Bagging, or Bootstrap Aggregating, reduces variance by training multiple decision trees on bootstrap samples of the training data. If B trees are trained, the final prediction is obtained through majority vote:

$$\hat{y}(x) = \text{mode}\{h_1(x), h_2(x), \dots, h_B(x)\}$$

Bagging improves stability because averaging multiple high-variance learners reduces sensitivity to fluctuations in the training data.

4.3 Random Forest

Random Forest extends bagging by introducing random feature selection at each split. Instead of considering all features at each split, only a random subset is evaluated. This reduces correlation among trees and usually improves generalization performance.

The final prediction is also obtained through majority vote:

$$\hat{y}(x) = \text{mode}\{h_1(x), h_2(x), \dots, h_T(x)\}$$

5 Experimental Setup

All three models were trained and evaluated using the same processed dataset and the same train/test split to ensure fair comparison.

Hyperparameters were tuned using cross-validation with a reduced but representative search space. At least two hyperparameters were tuned for each model. The main hyperparameters considered included maximum tree depth, minimum samples per leaf, minimum samples per split, number of estimators, maximum samples for bagging, and maximum features for random forest.

The models were evaluated on the final test set using the following metrics:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

Training time and inference time were also recorded to compare computational behavior.

6 Results

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Train Time
Decision Tree	0.8615	0.7872	0.5774	0.6662	0.8988	80.94
Bagging	0.8615	0.7566	0.6210	0.6822	0.9126	631.01
Random Forest	0.8666	0.7893	0.6039	0.6843	0.9180	387.01

Table 1: Performance comparison of the three models on the final test set.

The Random Forest achieved the best overall performance, with the highest accuracy, F1-score, and ROC-AUC. Bagging improved over the single Decision Tree, especially in F1-score and ROC-AUC, which suggests better generalization.

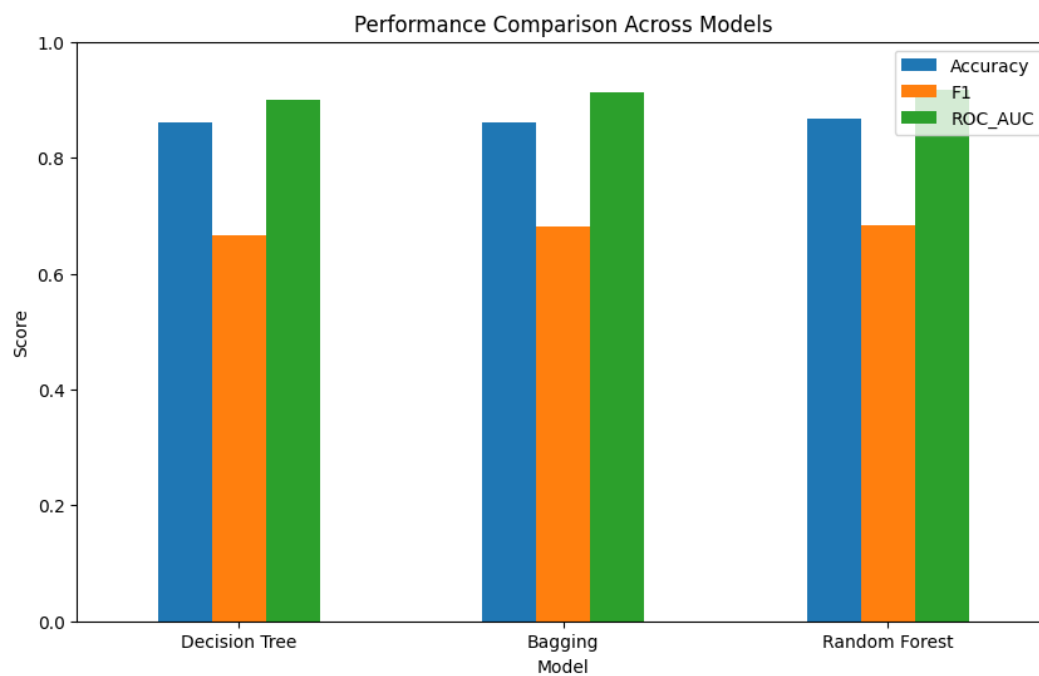


Figure 1: Performance comparison across Decision Tree, Bagging, and Random Forest using Accuracy, F1-score, and ROC-AUC.

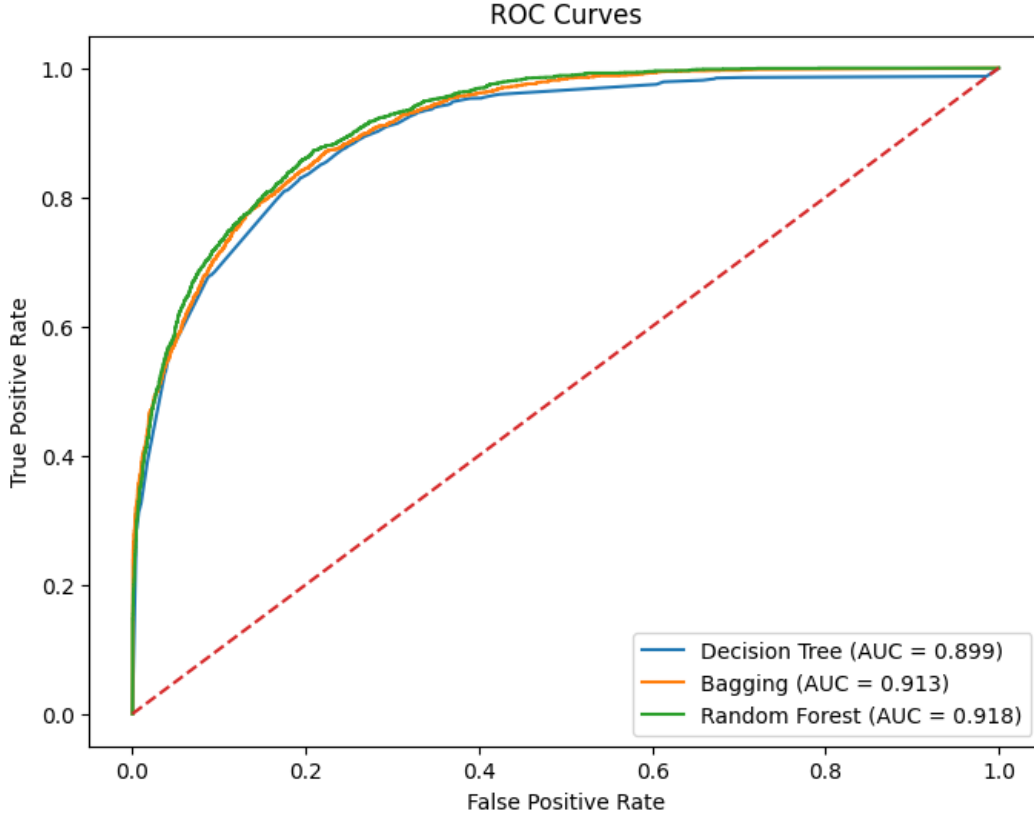


Figure 2: ROC curves for the three models on the test set.

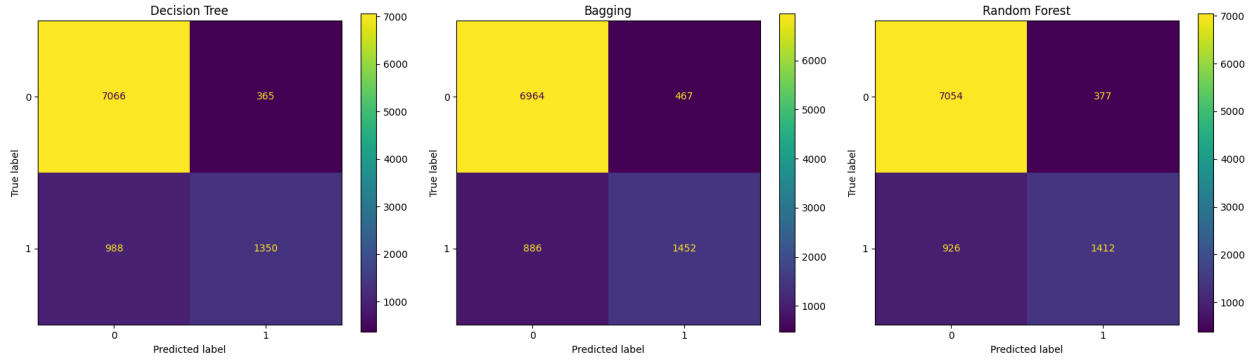


Figure 3: Confusion matrices for Decision Tree, Bagging, and Random Forest.

7 Deep Comparative Discussion

The Random Forest achieved the best generalization performance, supported by its highest F1-score of 0.6843 and highest ROC-AUC of 0.9180. These results indicate that it provided the strongest balance between classification accuracy and discrimination ability on unseen data.

The single Decision Tree showed weaker generalization than the ensemble methods. Although its accuracy remained competitive, its lower F1-score and ROC-AUC suggest that it was less effective at balancing false positives and false negatives. This is consistent with the tendency of decision trees to overfit the training data when model complexity increases.

Bagging improved performance relative to the single Decision Tree. Its F1-score increased from 0.6662 to 0.6822, and its ROC-AUC improved from 0.8988 to 0.9126. These improvements occurred because bagging reduces variance by averaging predictions from multiple bootstrap-trained trees.

Random Forest slightly outperformed Bagging. The main reason is that random feature selection at each split reduces correlation among trees, which increases ensemble diversity and leads to better generalization.

From a computational perspective, Bagging had the highest training cost, followed by Random Forest, while the Decision Tree was the most computationally efficient. This reflects the additional cost of training many trees in ensemble methods.

The main hyperparameters affected model complexity and performance in important ways. Increasing tree depth made individual trees more flexible but also increased the risk of overfitting. Increasing the number of estimators generally improved performance and stability for the ensemble methods, although gains began to level off beyond a certain point.

Random Forest appeared to be the most robust method because it combined strong test performance with improved variance reduction and resistance to overfitting. Bagging also improved robustness relative to the single tree, but Random Forest produced the best overall results.

From a bias-variance perspective, the Decision Tree had low bias but high variance. Bagging reduced variance by averaging multiple trees. Random Forest reduced variance even further by decorrelating the trees through feature subsampling.

Interpretability decreased as the models became more complex. The single Decision Tree was the most interpretable because its decisions can be traced directly through its branches. Bagging and Random Forest achieved better performance, but their predictions are harder to interpret because they aggregate many trees.

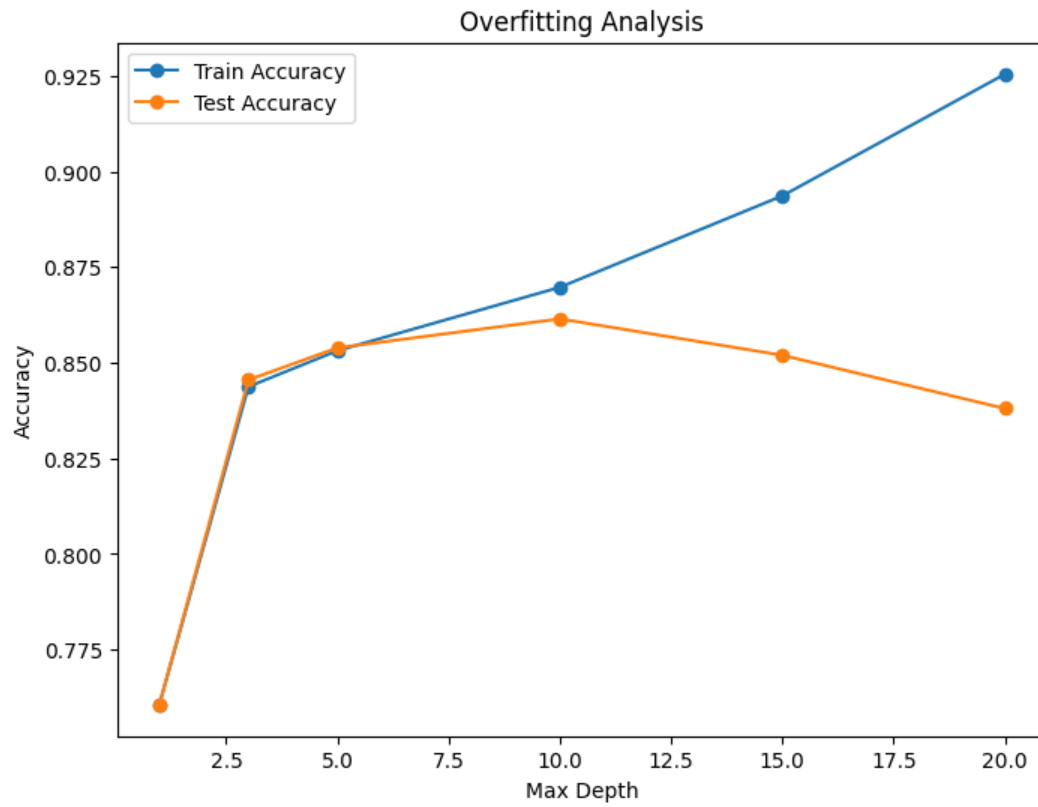


Figure 4: Overfitting analysis showing training and test performance as tree depth increases.

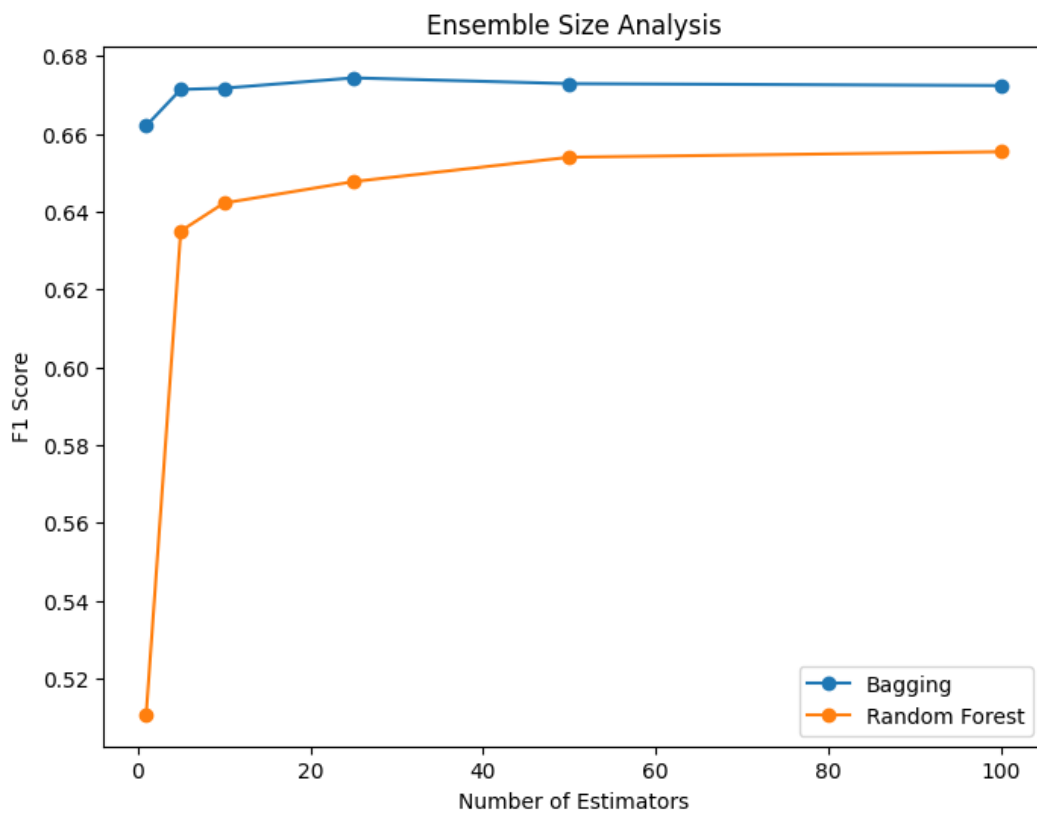


Figure 5: Effect of ensemble size on performance for Bagging and Random Forest.

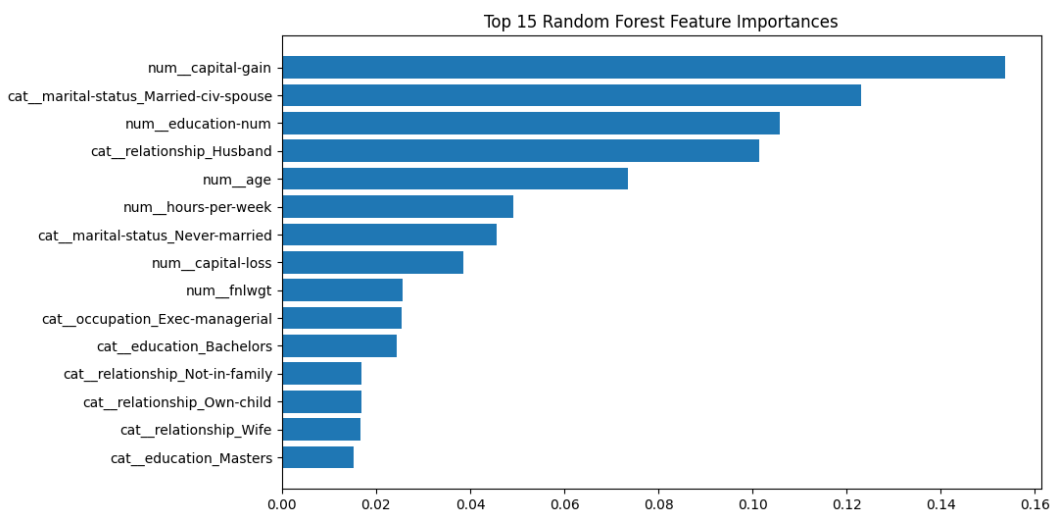


Figure 6: Top Random Forest feature importances.

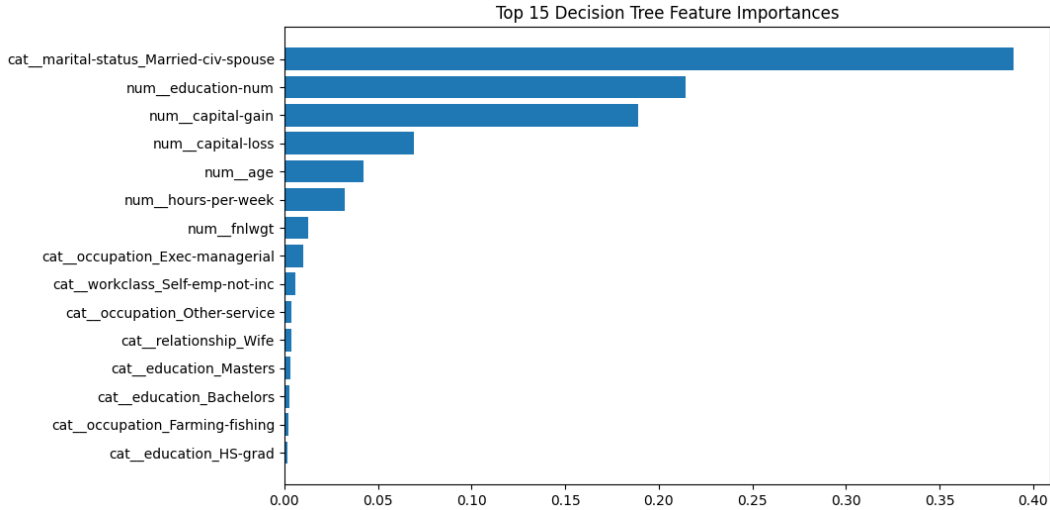


Figure 7: Top Decision Tree feature importances.

7.1 Practical Use Cases

A single Decision Tree is preferred when interpretability is the main priority and simple decision rules are needed.

Bagging is useful when reducing variance is important and one wants better stability than a single tree without fundamentally changing the base learner.

Random Forest is preferred in real-world applications where predictive performance, robustness, and resistance to overfitting are more important than simple interpretability.

8 Conclusion

This project compared a single Decision Tree, Bagging, and Random Forest on the Adult dataset. The results showed that ensemble methods improved generalization performance over a single decision tree.

Bagging reduced variance and improved predictive stability, while Random Forest further improved performance by reducing correlation among trees through random feature selection.

Among the three models, Random Forest achieved the best overall balance of predictive performance and robustness, making it the preferred method for this classification task.

9 References

UCI Machine Learning Repository. Adult Dataset.

Scikit-learn Documentation.

Course notes and lecture materials on Decision Trees, Bagging, and Random Forest.